모든 게시물을 가져오는 서비스 만들기 (CRUD => R)

모든 게시물 데이터를 데이터베이스에서 가져오는 로직 구현

이 로직은 Service에서 구현해주면 됩니다. 바로 데이터베이스와 연동해서 하면 되지만

처음부터 데이터베이스 연결해서 하면 헤깔릴수있기 때문에 우선 데이터를 로컬 메모

```
import { Injectable } from '@nestjs/common';

@Injectable()
export class BoardsService {
  private boards = [];
}
```

boards 배열안에 게시물 데이터들을 저장하겠습니다.

여기서 private을 쓴 이유는 만약 private을 사용하지 않는 다면 다른 컴포넌트에서 이 BoardsService에 접근해서 이 boards 배열 값을 수정할 수 있기 때문입니다.

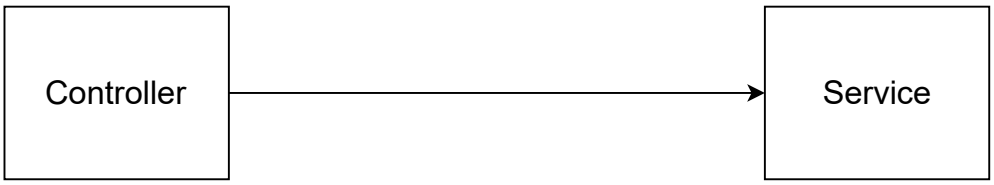
```
import { Injectable } from '@nestjs/common';

@Injectable()
export class BoardsService {
  private boards = [];

  getAllBoards() {
    return this.boards;
  }
}
```

boards 변수 안에 들어 있는 모든 게시물 데이터를 가져오기 위해서는 getAllBoards 메소드를 이용하면 됩니다. 그러면 이 메소드를 Call 하려면 어떻게 해야 할까요?

클라이언트에서 요청을 보내면 먼저 Controller를 타고 와서 Controller에서 요청에 맞게 라우팅을 해줍니다.



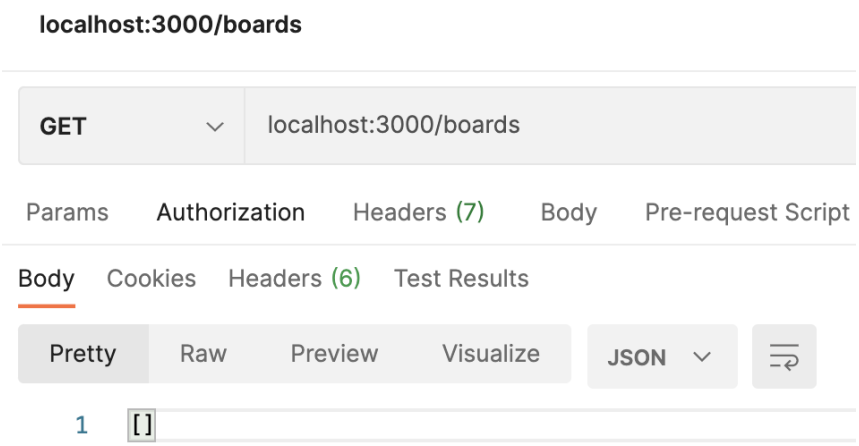

```
@Controller('boards')
export class BoardsController {
  constructor(private boardsService: BoardsService) {}

  @Get()
  getAllTask() {
    return this.boardsService.getAllBoards();
  }
}
```

클라이언트에서 요청을 보낼때 axios.get('/boards') 이런식으로 경로를 보내면 getAllTask 메소드에 요청이 도달하게 됩니다. Express 에서 보면 아래와 같습니다.

```
app.get('/boards', (req, res) => {
});
```

실제로 포스트맨을 이용해서 이 부분에 요청을 보내 보겠습



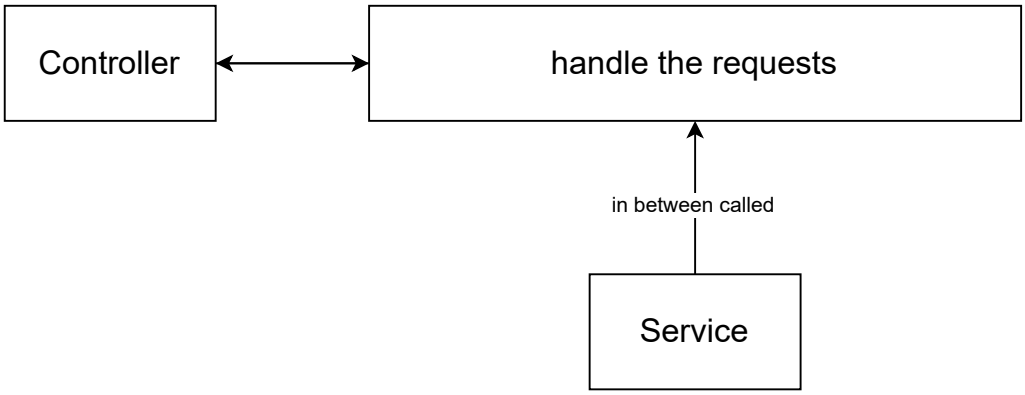
정리

클라이언트에서 요청을 보내면 먼저 컨트롤러로 가며 컨트롤러에서 알맞은 요청 경로

에

라우팅해서 해당 핸들러로 가게 해줍니다. 그런 후에 요청을 처리해주기 위해서 서비스로

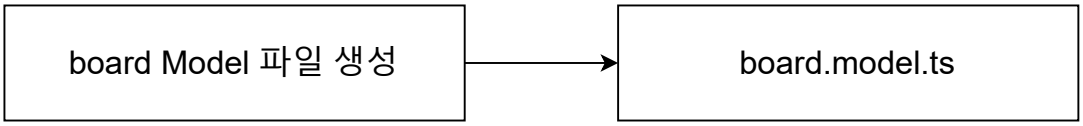
들어가며 그 요청에 맞는 로직을 서비스에서 처리해준 후 컨트롤러에 리턴값을 보내주후 컨트롤러에서 클라이언트로 결과값을 보내줍니다



Board Model 정의하기

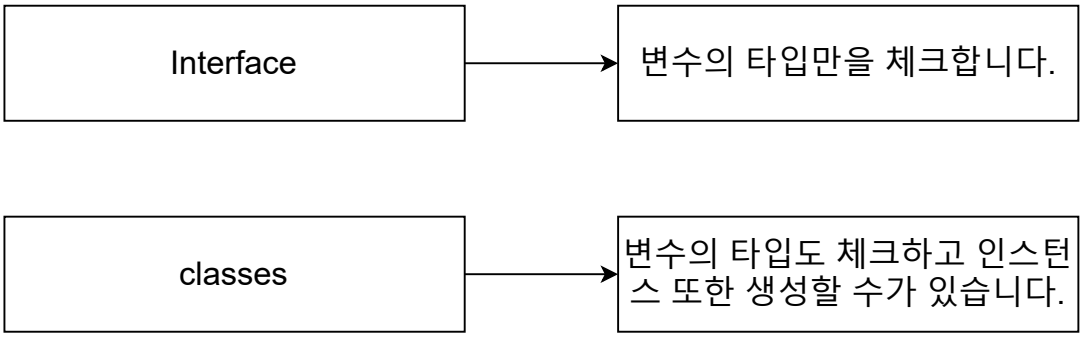
게시물 생성 기능을 만들기 전에 ....

게시물에 필요한 데이터가 어떤것이 필요한지를 정의해주기 위해서 게시물의 모델을 만들어주겠습니다. 예를들어 게시물 데이터에는 ID가 필요하고 이름이나 설명등이 필요하다....



모델을 정의하기 위해서는

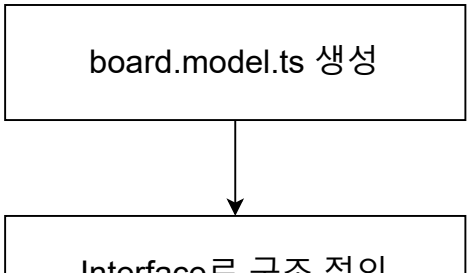
Class를 이용하거나 Interface를 이용하면 됩니다.



우선 board의 구조만 정의하기 위해서..

Interface를 사용하겠습니다...

board 모델 만들기



interface로 구조 정의

```
export interface Board {
  id: string;
  title: string;
  description: string;
  status: ;
}
```

BoradStatus란...?

이 게시물이 공개 게시물인지 아니면 비밀 게시물인지 나눠주는 것

그리고 이 두가지 상태 이외에는 나오면 안되기 때문에 이 두가지의 상태만 나올수 있게 하기 위해서 타입스크립트의 기능은 enumeration을 이용하겠습

```
export enum BoardStatus {
  PUBLIC = 'PUBLIC',
  PRIVATE = 'PRIVATE',
}
```

이렇게 status에 enum을 이용해서 정의한 BoardStatus를 넣어주면 BoardStatus에서 넣어준 PUBLIC이나 PRIVATE상태만을 사용할 수 있습니

```
export interface Board {
  id: string;
  title: string;
  description: string;
  status: BoardStatus;
}
```

생성한 Board 모델을 이용해서 타입을 정의해주겠습니다.

Service에 이미 만들어 놓은 boards 변수와 메소드에서 리턴해주는 값에 타입을 줍니다.

```
@Injectable()
export class BoardsService {
  private boards: Board[] = [];

  getAllBoards(): Board[] {
    return this.boards;
  }
}
```

Controller 에서 핸들러에서 결과값을 리턴해주는 부분도 타입을 정의해주겠습니다.

```
@Controller('boards')
export class BoardsController {
  constructor(private boardsService: BoardsService) {}

  @Get()
  getAllTask(): Board[] {
    return this.boardsService.getAllBoards();
  }
}
```

이렇게 타입을 정의해주면 좋은 이유?

타입 정의해주는 것은 선택사항입니다.

하지만 이렇게 타입을 정의해주므로서 원하는 타입과 다른 코드를 사용할 시 에러가 발생합니다.

그리고 코드를 읽는 입장에서 더 코드를 쉽게 이해하며 읽을 수 있습니다.

(readable)

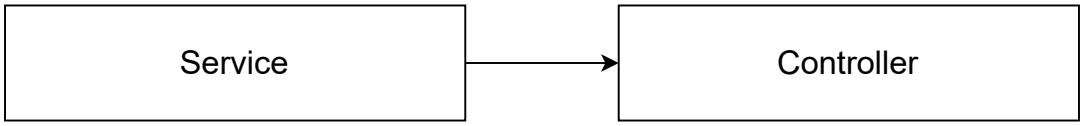
게시물 생성하기 (CRUD => C) : Service

게시물 생성 기능 만들기

게시물에 대한 모델을 정의해줬기에 이제는 게시물 생성하는 기능을 만들

기능을 만들기 위해서는 ...

게시물에 관한 로직을 처리하는 곳은 Service입니다. 그래서 먼저 Service 에서 로직을 처리해준 후에 Controller에서 서비스를 불러와주겠습니다.




```
createBoard(title: string, description: string) {
  const board: Board = {
    title,
    description,
    status: BoardStatus.PUBLIC,
  };
}
```

게시물 ID는 어떻게 처리하나요?

ID는 모든 게시물에 유니크 해야합니다. 그래서 만약 데이터베이스에 데이터를 넣어줄 때는 데이터베이스가 알아서 유니크한 값을 줍니다. 하지만 현재는 데이터베이스를 안쓰기 때문에 임의로 유니크한 값을 줘야합니다. 이때 여러 방법을 쓸 수 있지만 uuid 모듈을 이용해서 유니크한 값을 주겠습

uuid 모듈 사용하기 위해서

npm install uuid --save 를 이용해서 설치해줍니다.
그 후에 uuid 모듈을 사용하기 원하는 곳에서 import 해줍니다.
import { v1 as uuid } from 'uuid';

```
createBoard(title: string, description: string): Board {
  const board: Board = {
    id: uuid(),
    title,
    description,
    status: BoardStatus.PUBLIC,
  };

  this.boards.push(board);
  return board;
}
```

게시물 생성하기 (CRUD => C) : Controller

로직 부분을 처리했다면 이제 Request와 Response 부분 처리

request와 response 부분 처리는 Controller 에서 해주면 됩니다

```
@Post()
createBoard() {

}
```

클라이언트에서 보내온 값들은 핸들러에서 어떻게 가져오나요?

Express에서는 bodyParser 모듈을 이용해서
req.body 이런식으로 클라이언트에서 보내온 값을 받아왔습니다.

```
app.post('/', (req, res) => {
    console.log(req.body)
});
```

NestJS에서는 @Body body를 이용해서 가져옵니다.

이렇게 하면 모든 request에서 보내온 값을 가져올 수 있으며, 하나씩 가져오려면
@Body('title') title 혹은 @Body('description') description 이런식으로 가져오시면 됩니다

```
@Post()
createBoard(@Body() body) {
  console.log('body', body);
```

```
  }
```

```
@Post()
createBoard(
  @Body('title') title: string,
  @Body('description') description: string,
) {
  console.log('title', title);
  console.log('description', description);
}
```

이제는 Service에서 로직 처리를 할 수있게 값을 보내줍니다.

Request에서 보내 온 값들을 Controller에서 받아오면 이 값들을 이용해서 Service에서 로직 처리를 해줘야 합니다. 그래서 param으로 넘겨줍니다.

```
@Post()
createBoard(
  @Body('title') title: string,
  @Body('description') description: string,
): Board {
  return this.boardsService.createBoard(title, description);
}
```

이렇게 기능을 구현해줬으면 POSTMAN으로 테스트하겠습니다~

Data transfer Object(DTO)

DTO (Data Transfer Object)는 무엇인가요?

계층간 데이터 교환을 위한 객체입니다.

DB에서 데이터를 얻어 Service나 Controller 등으로 보낼 때 사용하는 객체를 말합니다.

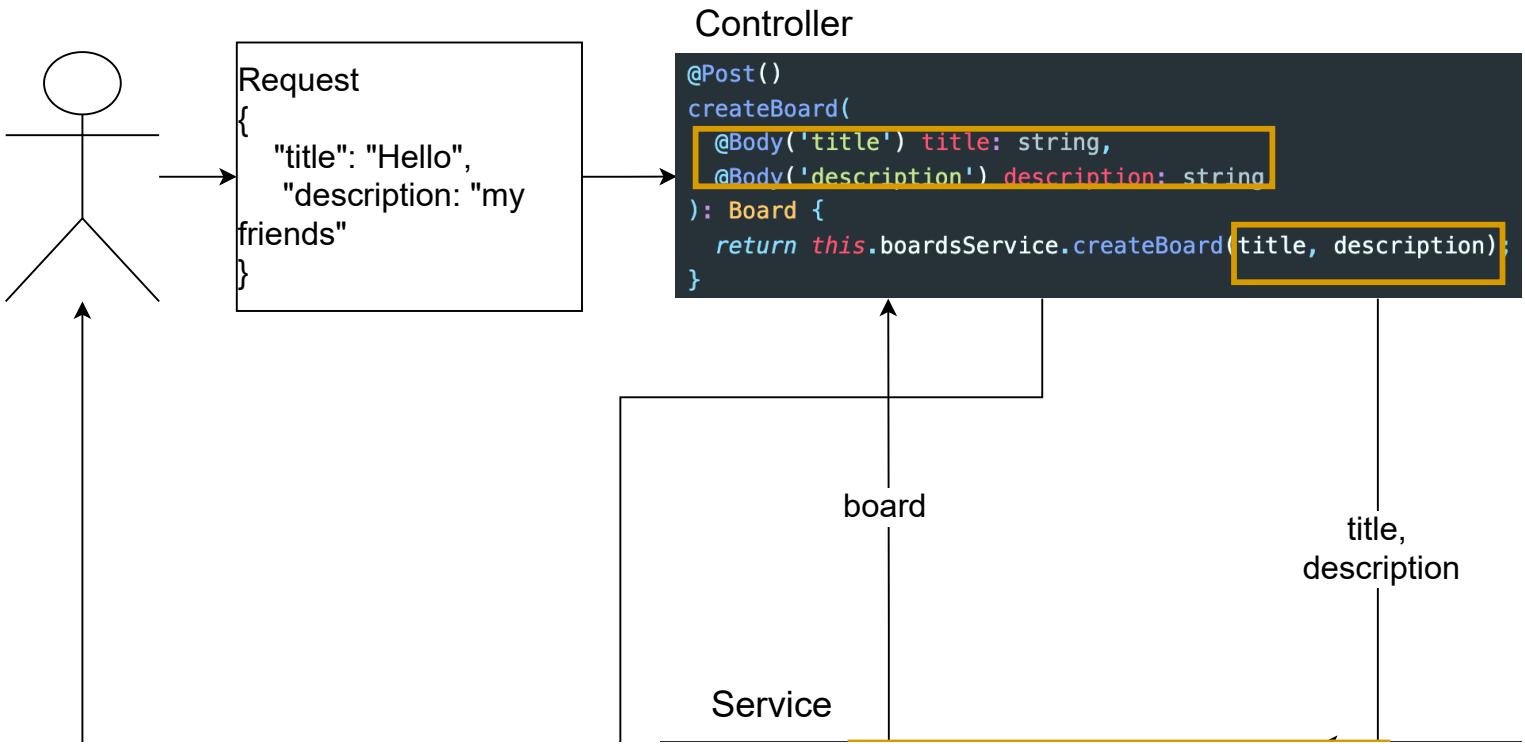
DTO는 데이터가 네트워크를 통해 전송되는 방법을 정의하는 객체입니다.

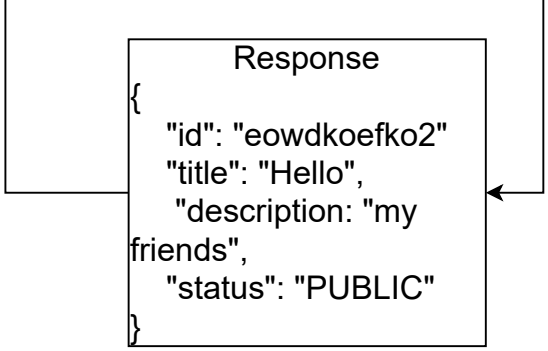
interface나 class를 이용해서 정의 될 수 있습니다. (하지만 클래스를 이용하는것을 Nest JS에서는 추천하고 있습니다)

DTO (Data Transfer Object)를 쓰는 이유는 무엇인가요?

- 데이터 유효성을 체크하는데 효율적입니다.

- 더 안정적인 코드로 만들어 줍니다. 타입스크립트의 타입으로도 사용됩니다.






```
createBoard(title: string, description: string): Board {
  const board: Board = {
    id: uuid(),
    title,
    description,
    status: BoardStatus.PUBLIC,
  };

  this.boards.push(board);
  return board;
}
```

Nest JS에서 데이터 처리의 흐름을 보면...

Board를 위한 Property들을 여러 곳에서 사용하고 있습니다. (title, description..)

지금은 간단한 애플리케이션을 만들기 때문에 몇개의 프로퍼티만 불러 주고 몇군데에서만 불러 주면 됩니다. 하지만 정말 많은 프로퍼티를 갖고 정말 여러군데에서 이용하며 갑자기 한곳에서 Property 이름을 바꿔줘야한다면 어떻게 해야할까요?
그러면 다른 곳에 똑같이 쓰인 모든 곳의 프로퍼티도 똑같이 바꿔줘야 합니

Interface VS Class For DTO

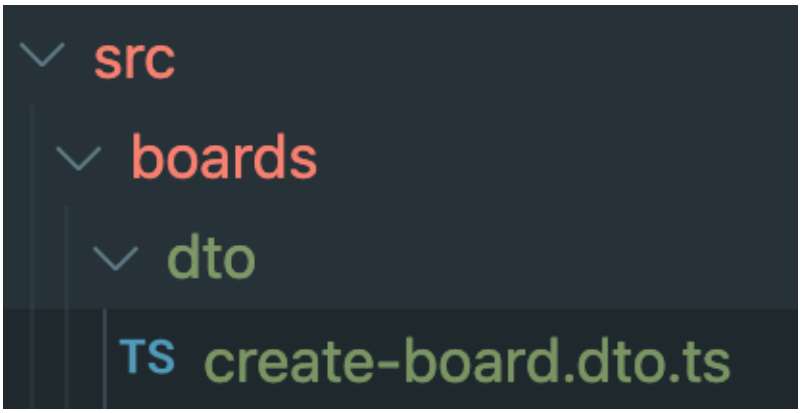
DTO 는 Interface나 Class를 사용해서 만들면 됩니다. 하지만 Class가 더 선호됩니다. 선호되는 이유는 ?

We could determine the DTO schema by using TypeScript interfaces, or by simple classes. Interestingly, we recommend using classes here. Why? Classes are part of the JavaScript ES6 standard, and therefore they are preserved as real entities in the compiled JavaScript. On the other hand, since TypeScript interfaces are removed during the transpilation, Nest can't refer to them at runtime. This is important because features such as Pipes enable additional possibilities when they have access to the metatype of the variable at runtime.

TypeScript 인터페이스를 사용하거나 간단한 클래스를 사용하여 DTO 스키마를 결정할 수 있습니다. 흥미롭게도 여기에서 Class를 사용하는 것이 좋습니다. 왜? 클래스는 JavaScript ES6 표준의 일부이므로 컴파일 된 JavaScript에서 실제 엔티티로 유지됩니다. 반면에 TypeScript 인터페이스는 트랜스 파일 중에 제거되므로 Nest는 런타임에서 참조 할 수 없습니다.
이것은 파이프와 같은 기능을 런타임에서 사용할 수 있기 때문에 런타임에서 사용될 수 있는게 중요합니다. 그래서 DTO는 CLASS를 이용해서 만들겠습니다.

게시물 생성을 위한 DTO

전 강의에서 DTO에 대해서 알아보았기 때문에
이번 시간에는 게시물 생성을 위한 DTO를 실제로 생성해보겠습니다



DTO 파일 작성

클래스는 인터페이스와 다르게 런타임에서 작동하기 때문에 파이프 같은 기능을
이용할 때 더 유용합니다. 그래서 클래스를 사용해서 DTO를 작성합니다.

```
export class CreateBoardDto {
  title: string;
  description: string;
}
```

DTO 적용하기

이렇게 DTO를 만들었으면 실제 Controller와 Service에서 Dto를 적용하겠습니다.

```
@Post()
createBoard(@Body() createBoardDto: CreateBoardDto): Board {
```

```
  return this.boardsService.createBoard(createBoardDto);
}
```

Service를 위해서도 DTO 적용

```
createBoard(createBoardDto: CreateBoardDto): Board {
  const { title, description } = createBoardDto;

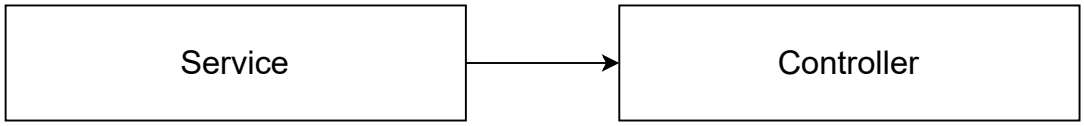
  const board: Board = {
    id: uuid(),
    title,
    description,
    status: BoardStatus.PUBLIC,
  };
```

이렇게 해서 Controller와 Service에 DTO를 적용해주었습니다.
여기서는 그냥 Build 하는 시간에 타입을 체크하는 부분만 해줬지만
이후에는 이것과 파이프라는것을 이용해서 런타임에서
유효성 체크등 다른것들도 해줄것입니다.

ID로 특정 게시물 가져오기

이번에는 ID를 이용해서 특정 게시물을

가져오는 기능을 구현하겠습니다.



Service

```
getBoardById(id: string): Board {
  return this.boards.find((board) => board.id === id);
}
```

Controller

```
@Get('/:id')
getBoardById(@Param('id') id: string): Board {
  return this.boardsService.getBoardById(id);
}
```

```
import Param
```

Route handler parameter decorator. Extracts the `params` property from the `req` object and populates the decorated parameter with the value of `params`. May also apply pipes to the bound parameter.

For example, extracting all params:

```
findOne(@Param() params: string[])
```

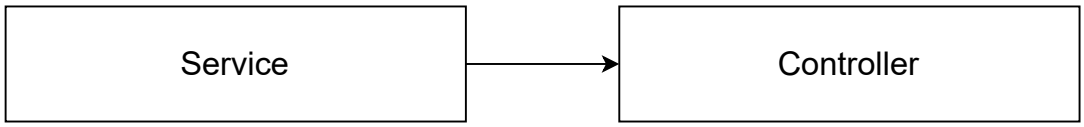
For example, extracting a single param:

```
findOne(@Param('id') id: string)
```

ID로 특정 게시물 지우기

이번에는 ID를 이용해서 특정 게시물을

삭제하는 기능을 구현하겠습니다.



Service

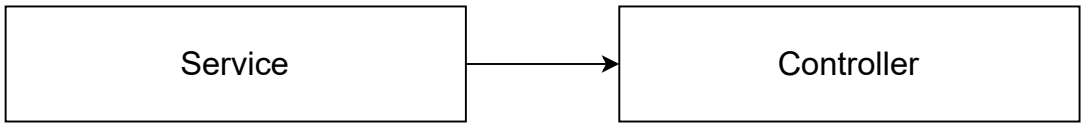
```
deleteBoard(id: string): void {
  this.boards = this.boards.filter((board) => board.id !== id);
}
```

Controller

```
@Delete('/:id')
deleteBoard(@Param('id') id: string): void {
  this.boardsService.deleteBoard(id);
}
```

특정 게시물의 상태 업데이트

특정 게시물의 상태(PUBLIC 또는 PRIVATE)을 업데이트 해주는



Service

```
updateBoardStatus(id: string, status: BoardStatus): Board {
  const board = this.getBoardById(id);
  board.status = status;
  return board;
}
```

Controller

```
@Patch('/:id/status')
updateBoardStatus(
  @Param('id') id: string,
  @Body('status') status: BoardStatus,
) {
  return this.boardsService.updateBoardStatus(id, status)
}
```

we created a handler in our controller and link the batch request we're extracting

the euro and body parameters here and then we're calling the service update desk status method and returning

the result which is a desk service method accepts an idea which is a string any status which is a task

status retrieving the already existing desk from the array we're having reference to it in this static

variable updating the status and then returning it back and if you're worried about validation not existing

tasks may be some invalid status provided and don't worry we have an entire section dedicated to making

our application more bulletproof by adding some validation and error handling in the future.